Capstone Project Guide

Build a working web application that solves a real problem for a clearly defined audience. This capstone brings together the skills developed across the bootcamp: command line tools, Git, Python, HTML and CSS, Django, relational databases, and the Django ORM.

Your project and its purpose

You are free to choose your idea. Your application must have a credible path to market: someone should have a practical reason to use it, pay for it, or adopt it within an organization. Existing ideas are welcome when you can explain the audience, value, and reason for choosing your solution.

Develop a complete, functional web application that addresses your chosen problem and delivers the features within your agreed scope. The application should be reliable, usable, and suitable for presentation to potential users or customers. Your final demonstration must show how it works in a realistic scenario and why someone would choose to use it.

What market potential means

- A specific audience with a recognizable problem or unmet need.
- A useful workflow that works from beginning to end with persisted data.
- A clear reason to choose the product over an existing tool or manual process.
- A realistic adoption, revenue, or organizational value model.
- A manageable scope that you can build, verify, and explain.

A commercial launch, paying customers, or a completely original invention is not required. Distinguish evidence from assumptions when describing demand or benefits. A polished landing page alone does not demonstrate a working product.

How to use this guide

Develop the idea discussed with the instructor and work through the checkpoints in this guide. This is an individual project. Each student must develop, submit, and demonstrate their own application. Project dates and presentation slots will be announced separately. Assessment totals 100 marks.

Examples of suitable scope

Examples include equipment booking for a training center, custom order tracking for a small shop, maintenance requests for a property manager, or appointment management for a local service. These are starting points; your own idea is welcome.

For example, an equipment booking application could let users find equipment, request a date range, and track a decision. Staff could review requests and see demand by equipment. Payments, mobile apps, and smart locks could remain future enhancements.

Build the Django application

Application structure

- Use Python and Django for the application backend, with a relational database managed through Django models and migrations.
- Organize apps, URLs, views, templates, and static files clearly. Use URL names and namespaces where appropriate.
- Use template inheritance for shared layouts and reusable components where they improve consistency.
- Keep views readable. Organize repeated logic into suitable functions or modules rather than duplicating it.
- Provide a clear route through the application so users can discover and complete the core workflow.

Working features

- Implement the approved workflow through the application interface and save its data in the database.
- Provide relevant create, read, update, and delete operations for the main records. Use cancellation, archiving, or restricted deletion when the business rules require it.
- Include useful search or filtering and an intentional ordering of results. Use pagination where the list size warrants it.
- Validate required fields, acceptable values, and business rules on the server. Display helpful feedback for success and failure.
- Handle missing records, empty lists, and invalid requests without exposing an unhandled error page during normal use.

Usable interface

- Use semantic HTML, associated form labels, and meaningful alternative text for content images.
- Create a responsive layout that remains usable on desktop and mobile widths.
- Maintain consistent navigation, spacing, typography, and action labels.
- Show useful empty states, validation messages, and clear confirmation of completed actions.
- Confirm destructive actions and make the consequences understandable.

Accounts and access

If your workflow uses accounts or different roles, define what each role may view and change, and enforce those rules on the server. Use Django authentication rather than storing passwords yourself. A public demonstration must not expose real personal information or credentials.

Additional features

Choose uploads, authentication, APIs, notifications, external integrations, or deployment when they serve the agreed scope and you can implement them responsibly. They are not universal requirements. Use a clearly labeled simulation where a real integration is unavailable; demonstrate the core application independently of it.

Design and use your data

Database design

- Create an ERD showing entities, primary keys, foreign keys, relationship cardinality, and optionality.
- Translate the ERD into models with appropriate field types, lengths, defaults, and required or optional values.
- Avoid repeating groups and unnecessary duplication. Be ready to explain where information belongs and why.
- Use a foreign key relationship and enough related entities to support the approved workflow. Add one to one or many to many relationships only when justified.
- Apply suitable uniqueness rules, constraints, and deletion behavior to protect important business rules.
- Include migrations in the repository and verify that they build the schema from a fresh setup.

ORM requirements

Demonstrate these capabilities in working application features. Identify where each appears in your submission; queries used only in an isolated shell session do not establish integration.

Capability	Application evidence
CRUD	Create, retrieve, update, and safely remove or retire relevant records.
Filtering and ordering	Find useful subsets of records and display them in a sensible order.
Relationship queries	Retrieve or filter records through a related model.
Aggregation	Calculate at least one useful summary across a queryset, such as a total, count, or average.
Annotation	Calculate at least one useful value per result, such as requests per item or orders per customer.

Make reporting useful

Display the summaries in a relevant page or report. Explain what decision the information supports. For example, a maintenance application could show a total of open requests and annotate each location with its unresolved request count.

Data quality and query correctness

Use realistic sample data that includes multiple related records and useful edge cases. Check that totals, counts, and averages remain correct when relationships contain several matching rows. Explain the meaning of each metric and how empty results are handled.

You may use Q objects, F expressions, transactions, select_related, or prefetch_related where they solve a real need. Advanced syntax does not earn marks by itself; correct behavior and clear reasoning matter.

Work through the checkpoints

Checkpoint	Evidence to present
Design	ERD, simple wireframes, user journeys, and a prioritized task list.
First working workflow	One complete journey through the interface with saved data and validation.
Feature completion	Remaining core features, useful reports, and resolved integration issues.
Final readiness	Fresh setup verified, acceptance checks completed, documentation and demo prepared.

Plan the work

Break features into small tasks with a clear completion condition. Build a complete workflow early, then expand it. Keep an updated list of completed work, remaining work, and blockers.

Verify the application

Maintain a short verification record with the scenario, expected result, actual result, and status. Fix failures or list them honestly as known limitations. Include checks for:

- The main workflow from the first action to the final saved outcome.
- Missing required fields and invalid business values.
- Search and filters with matching results and with no results.
- Updates and deletion or cancellation of records that have dependencies.
- Aggregation and annotation against sample data with known expected results.
- Access restrictions if accounts or roles are used.
- Mobile layout and an installation from a fresh checkout.

Manual acceptance checks are required. Automated tests are encouraged for important rules and calculations, but no particular test framework or coverage percentage is required.

Use tools with understanding

You may use documentation, libraries, tutorials, and AI assistance. Acknowledge significant external resources and assistance in the README. Review, adapt, and verify what you use. Every student must understand and explain their own implementation and decisions.

Use fictitious or anonymized demonstration data. Keep passwords, secret keys, tokens, and private files out of the repository. Document configuration through safe examples.

Submit and demonstrate your product

Submission package

- Source repository: include application code, templates, static files, migrations, and dependency information. Provide access to the instructor.
- README: explain the problem, target users, core features, setup steps, configuration, sample data, and how to run the application.
- Design material: Design material: include the final ERD and any wireframes needed to explain the application's workflow.
- Verification record: include acceptance scenarios, outcomes, known issues, and relevant automated tests if used.
- Demo information: explain how to reproduce the demonstration and access sample accounts if applicable. Include a deployment link only if deployed.

Reproducible setup

The README must allow someone else to create an environment, install dependencies, configure the application, apply migrations, load or create sample data, and start the server. State the Python and Django versions used. A fresh setup must work without relying on files or data available only on your machine.

Final demonstration

Prepare a concise demonstration in this order. The instructor will confirm the presentation duration.

- Introduce the intended users, problem, and evidence supporting the idea.
- Explain the value and show the main workflow with realistic data.
- Show one validation or failure case and the application's response.
- Present the relevant report, including aggregation and annotation.
- Explain key database relationships and one important technical decision.
- Describe market potential, current limitations, and the next practical improvement.

Individual technical discussion

Each student must explain their implementation and answer questions about their code and design decisions. Be ready to trace a request from the URL through the view, ORM, and template; explain a relationship or constraint; and show how a calculation or validation rule works.

Before submission

Confirm that the submitted repository version matches the demonstration. Remove unused files, check access permissions, run the documented setup, and verify that the core workflow still works. Report limitations accurately so the reviewer can assess the delivered product.

Assessment and final checklist

Assessment totals 100 marks. Working evidence, appropriate design, and technical understanding determine the score. Extra features add value only when they support the product and work reliably.

Criterion	Marks	What is assessed
Problem and market potential	10	Specific audience, evidence, clear value, realistic adoption or revenue model, and achievable scope.
Core functionality	20	Complete approved workflows, correct behavior, persistence, and useful error handling.
Database and ORM	20	ERD and models, relationships, integrity, query correctness, aggregation, and annotation.
Django and Python implementation	15	Clear structure, routing, views, templates, reusable logic, and readable code.
Interface and usability	10	Responsive layout, accessible forms, clear navigation, and helpful feedback.
Verification and reproducibility	10	Meaningful checks, working fresh setup, useful sample data, and honest limitations.
Git and documentation	5	Meaningful development history, usable README
Demonstration and individual understanding	10	Clear product demonstration and ability to explain personal work and technical decisions.

Final checklist

- The project solves a clear problem for an identifiable audience.
- The approved core workflow works from beginning to end.
- The ERD matches the implemented models and migrations.
- CRUD, filtering, relationship queries, aggregation, and annotation are demonstrated.
- Validation, empty states, and relevant failure cases have been checked.
- The interface works at desktop and mobile widths.
- The documented fresh setup works and includes suitable sample data.
- The repository contains no credentials or real private data.
- Deliverables are complete and each student can explain their work.

Build something focused, useful, and dependable. Your strongest demonstration is a product whose purpose is clear, whose essential workflow works, and whose implementation you can confidently explain.

Best of luck! See you in-class.